

DDLanch+ (Data-Driven Launcher+)

v2.1.0

A small, native Win32 C++ desktop launcher. It lists remote resources (hosts, drives, shell items, URLs) from a JSON file and opens each one with a tool chosen by its **type**. Which tool handles which type is also defined in JSON, so you can add new connection types without recompiling.

Author	Glenn Madine
Version	2.1.0 (release date 09/25/2026)
Language / API	C++17, Win32 API, Common Controls v6, COM (Shell.Application)
Compiler	Microsoft C++ 19.51 (MSVC)
Dependencies	Windows SDK, nlohmann/json (single header <code>json.hpp</code>), <code>resource.h</code> / <code>DDLanch.res</code>
Source file	<code>DDLanch.cpp</code> (~1,560 lines)
Minimum OS	Windows Vista or later; best on Windows 10 1703+ (Per-Monitor V2 DPI)

1. What it does

When started, DDLanch+ opens a resizable window, sized for the DPI of the monitor it opens on. The window contains:

- **A report-style list view** with three sortable columns: *Resource (Host, Drive, or Shell Item)*, *Type*, and *Description*.
- **Five buttons** in two rows, sized to fit their captions:

Button	Behavior
Launch RDP connection for host not in this list	Starts <code>mstsc.exe</code> with no arguments; reports an error if it can't start
Edit <code>actionDefinitions.json</code>	Opens the actions file in Notepad
Edit <code>connections.json</code>	Opens the connections file in Notepad
Run...	Shows the standard Windows Run dialog (same as Win+R)
Exit	Closes the application

Launching. Double-click a row, or press **Enter** on it. The app upper-cases the row's **Type** and looks it up in `actionDefinitions.json`:

- If the action's `command` starts with a URI scheme (`https://`, `file:///`, `ssh://`, `vnc://`, `mailto:`, `ms-settings:...`), the app opens `command + host` with the program registered for that scheme.
- Otherwise the app runs `command` as a program, with `args + host` as its parameters.

Error messages. The app shows a specific message when:

- the entry has no type
- no action matches the type (the message also says when the name matches except for upper/lower case)
- the action's command is empty
- the launch fails (the message includes the Windows error code and text)

Sorting. Click a column header to sort by that column, ignoring case. Click it again to reverse the order. The sort, and the arrow in the header, are kept when the data reloads.

Hot reload. When you switch back to the window, it reloads a JSON file only if the file's last-write time or size has changed. Edits made in Notepad show up right away, and an invalid file reports its error once.

Keyboard.

Key	Action
Tab / Shift+Tab	Move between the list and the buttons
Arrow keys	Move within the list
Enter	Launch the selected row, or press the focused button
Space	Press the focused button
Esc	Ignored on purpose, so it can't close the launcher

When the window is re-activated, focus goes back to the control that last had it.

High DPI. The app runs Per-Monitor V2 DPI aware. The font (the system message font, normally Segoe UI 9pt), column widths, buttons and margins scale with the monitor, and they update live when the window moves to another monitor. The window has a minimum size so the buttons never overlap.

2. Configuration files

Both files live in a per-user folder:

```
%USERPROFILE%\DDLlaunch\actionDefinitions.json
%USERPROFILE%\DDLlaunch\connections.json
```

If `%USERPROFILE%` is not set, the app uses the shell's Profile known folder instead. If that also fails, it uses the folder that contains `DDLlaunch.exe`. The folder and both files are created on first run. Existing files are never overwritten.

2.1 actionDefinitions.json

The file must be a top-level JSON **array**. Every entry needs all three keys, or the file fails to load (the app then keeps the actions it last loaded successfully).

Key	Meaning
action	Name matched against a connection's <code>type</code> . The type is upper-cased before matching, so write action names in UPPER CASE
args	Text placed before the host on the command line (e.g. <code>" /v:"</code>). Not used for URL commands
command	Path to a program, or a URL prefix with any scheme (e.g. <code>https://</code> , <code>vnc://</code>)

Default content written on first run:

```
[
  { "action": "RDP",    "args": " /v:", "command": "C:\\Windows\\System32\\mstsc.exe" },
  { "action": "SSH",    "args": " ",   "command": "C:\\Windows\\System32\\OpenSSH\\ssh.exe" },
  { "action": "HTTPS",  "args": "",    "command": "https://" },
  { "action": "HTTP",   "args": "",    "command": "http://" }
]
```

Example of a custom action, which needs no recompile:

```
{ "action": "VNC", "args": "", "command": "vnc://" }
```

How the app tells a URL from a program path: it looks for a scheme as defined by RFC 3986 (ALPHA *(ALPHA / DIGIT / "+" / "-" / ".") ":"). Schemes must be at least two characters long, so drive letters such as `C:\...` are still treated as program paths.

2.2 connections.json

The file can be a flat array of device objects, or an object of named groups (each group is an object or an array of objects). Each device can have `host`, `type`, and `description` (all strings, all optional). The default is an empty array `[]`.

```
[
  { "host": "dc01.ynext.corp", "type": "RDP", "description": "Domain controller" },
  { "host": "piweb.ynext.corp", "type": "HTTPS", "description": "Raspberry Pi web server" },
  { "host": "admin@pi01",      "type": "ssh", "description": "Pi shell (type is case-insensitive)" }
]
```

Grouped form (group names are not shown; all devices are merged into one list):

```
{
  "Servers": [ { "host": "ca1.ynext.corp", "type": "RDP", "description": "Certificate Authority" } ],
  "Web":     { "host": "intranet.ynext.corp", "type": "HTTPS", "description": "Intranet" }
}
```

3. Building

Requirements: Visual Studio (MSVC 19.x) with the Windows SDK, `json.hpp` from [nlohmann/json](#), and a `resource.h` / `DDLaunch.rc` that defines `IDI_ICON1`.

From a *Developer Command Prompt for VS*:

```
rc DDLaunch.rc
CL /EHsc /W3 /O2 /GL /DUNICODE /D_UNICODE /DNDEBUG /std:c++17 DDLaunch.cpp DDLaunch.res /Fe:DDLLaunch.exe
/link /SUBSYSTEM:WINDOWS /MANIFEST:EMBED comctl32.lib shell32.lib shlwapi.lib user32.lib gdi32.lib ole32.lib oleaut32.lib
```

Manifest. The source declares the Common Controls v6 dependency with `#pragma comment(linker, "/manifestdependency:...")`, and `/MANIFEST:EMBED` embeds the manifest in the .exe. If `DDLaunch.rc` also contains its own `RT_MANIFEST` resource, remove one of the two to avoid a duplicate-resource link error.

DPI. DPI awareness is set at run time by `Dpi::EnableDpiAwareness()`, so no manifest entry is needed for it. If a manifest does declare DPI awareness, the manifest wins.

4. Code architecture

Section	Key functions / types	Purpose
Header / constants	<code>VERSION</code> , <code>IDC_*</code> , <code>COL_*</code> , manifest pragma	Version, control IDs, column indexes, Common Controls v6 dependency
UTF-8 ↔ UTF-16	<code>Utf8ToWide</code> , <code>WideToUtf8</code>	Convert between JSON (UTF-8) and Win32 wide strings
Action model	<code>struct Action</code> , <code>from_json</code> , <code>to_json</code>	nlohmann/json mapping for action entries
File I/O	<code>ReadFileUtf8</code>	Reads a whole file via <code>CreateFileW/ReadFile</code> (supports Unicode paths)
Action lookup	<code>loadActions</code> , <code>findAction</code> , <code>findActionIgnoreCase</code>	Parse the actions array; a single lookup that returns <code>const Action*</code>
Device model	<code>struct Device</code> , <code>LoadConnections</code>	Parse connections (array or grouped object)
Paths	<code>GetUserProfileDir</code> , <code>GetDataDir</code> , <code>Get*JsonPath</code>	Resolve <code>%USERPROFILE%\DDLLaunch</code> , with fallbacks
First-run setup	<code>EnsureDataFiles</code> , <code>CreateFileWithDefaultContent</code>	Create the folder and write the default JSON (<code>CREATE_NEW</code> , safe if two instances race)
Editing	<code>OpenJsonFileForEditing</code>	Opens a config file in Notepad
URL detection	<code>StartsWithUrlScheme</code>	Generic RFC 3986 scheme check (ignores drive letters)
Launching	<code>ShellOpen</code> , <code>LaunchSomething</code> , <code>ReportLaunchFailure</code> , <code>GetErrorText</code>	<code>ShellExecuteExW</code> with <code>SEE_MASK_FLAG_NO_UI</code> ; clear error messages
Run dialog	<code>ShowRunDialog</code>	<code>IShellDispatch::FileRun()</code> via COM
DPI helpers	namespace <code>Dpi</code>	Enable awareness, get DPI per window, scale values, create the UI font, adjust window rect (newer APIs loaded dynamically)

Section	Key functions / types	Purpose
Sorting	<code>ApplyCurrentSort</code> , <code>ToggleSortColumn</code> , <code>UpdateHeaderSortArrow</code>	Stable sort that is re-applied after every reload
Change detection	<code>FileStamp</code> , <code>GetFileStamp</code> , <code>ReloadAllData(hwnd, force)</code>	Reload a file only when it changed; guarded against re-entry while an error box is open
Item activation	<code>OnItemActivated</code> , <code>ActivateSelection</code>	Validate type and action, then launch
Layout	<code>namespace Layout</code> , <code>S()</code> , <code>MeasureButtonWidth</code> , <code>ApplyDpi</code> , <code>CreateButton</code>	Sizes in 96-DPI units, scaled per monitor; buttons sized to fit their text
Window procedure	<code>WndProc</code>	Create, size, min/max size, DPI change, commands, notifications, activation/focus
Entry point	<code>wWinMain</code>	DPI awareness, common controls, window class, <code>IsDialogMessageW</code> message loop

Launch flow

1. The user double-clicks a row, or presses Enter on it. Enter arrives as `IDOK` via `IsDialogMessageW`, or as `NM_RETURN`.
2. `OnItemActivated` checks that the entry has a type, that an action exists for it (hinting about a case mismatch if needed), and that the command isn't empty.
3. `LaunchSomething` uses `StartsWithUrlScheme(command)` to choose one of two calls:
 - **URL:** `ShellExecuteExW("open", command + host)`
 - **Program:** `ShellExecuteExW("open", command, args + host)`
4. On failure, `ReportLaunchFailure` shows the host, type, command, parameters, and the Windows error code and text. A cancelled UAC prompt is not reported as an error.

Reload flow

1. `WM_CREATE` calls `ReloadAllData(hwnd, true)`, which loads both files.
2. On each `WM_ACTIVATE`, `ReloadAllData(hwnd, false)` compares the current `FileStamp` of each file with the stored one. If both match, it returns straight away.
3. For each file that changed, the new stamp is recorded **before** the file is parsed, so an error is shown once per save. Devices are re-sorted with the current sort, and the list is repopulated.

Strengths

- **Unicode throughout:** wide Win32 APIs, UTF-8 JSON, and Win32 file I/O, so profile paths with non-ASCII names work.
- **Data-driven:** a new launch type (including a custom URL scheme) needs only a JSON edit.
- **Clear feedback:** every failure path produces a specific message that says what to change and where.
- **Reload cost:** it happens only when a file changes, and it doesn't loop on errors.
- **Keyboard:** full keyboard operation, with tab order matching the layout and focus restored on activation.

- **High DPI:** Per-Monitor V2 with live rescaling, fallbacks for older Windows versions, and a minimum window size.
- **Documented APIs only:** uses `Shell.Application` for the Run dialog, and loads the newer DPI APIs dynamically so the .exe still starts on older systems.
- **Maintainable:** a version-history comment block in the header records each change.

5. Analysis: remaining issues and recommendations

Severity: **Medium** = incorrect or fragile behavior; **Low** = cleanup, maintainability, or minor UX.

5.1 Behavior

#	Severity	Issue	Recommendation
B5	Medium	Action names are still matched case-sensitively. Types are upper-cased, but an action written as "Rdp" never matches. Since 2.0.3 the error message points this out, but the lookup still fails.	Match with <code>_wcsicmp</code> in <code>findAction</code> (then <code>findActionIgnoreCase</code> can be removed).
B8	Low	Connections are cleared on a read error. If <code>connections.json</code> can't be read or parsed, the list becomes empty, while a bad actions file keeps the old actions.	Keep the previous <code>g_devices</code> when loading fails, matching the actions behavior.
B9	Low	Selection is lost on reload and sort. <code>PopulateListView</code> deletes all items, so the selected row (and the keyboard focus item) resets after a column click or a reload.	Remember the selected Device (host + type) before repopulating and reselect it with <code>ListView_SetItemState</code> + <code>ListView_EnsureVisible</code> .
B10	Low	Garbled non-ASCII text in JSON error messages. <code>MessageBoxA(e.what())</code> shows UTF-8 text (which can include the file path) in the ANSI code page, so a profile name such as <code>C:\Users\José</code> displays incorrectly.	Convert with <code>Utf8ToWide(e.what())</code> and use <code>MessageBoxW</code> .
B11	Low	Host values are appended unquoted. A host containing spaces is split into separate command-line arguments.	Quote the host when it contains spaces (for non-URL commands).
B12	Low	Entries without a host are still listed. They show as blank rows and only fail on launch.	Skip them when loading, or show them with a warning marker.

5.2 Dead or duplicated code

#	Item	Notes
D1	<code>UsernameDialog</code> struct and <code>GetUsernamePopup()</code>	Never called; they contain unused locals (<code>dlgBase</code> , <code>DlgLayout</code>). Remove them, or use them for SSH (e.g. prompt for <code>user@host</code>).

#	Item	Notes
D2	<code>class Logger</code>	Never instantiated. Remove it, or use it to log launches and errors.
D3	<code>getExeDirectory()</code> vs <code>GetExeDir()</code>	Duplicates; only <code>GetExeDir()</code> is used.
D4	<code>ToWide()</code> vs <code>Utf8ToWide()</code> ; <code>WStringToString()</code> vs <code>WideToUtf8()</code>	Duplicate conversion helpers; <code>WStringToString</code> is unused.
D6	<code>PopulateListView(hLV, actions),</code> <code>OnItemActivated(..., actions)</code>	The <code>actions</code> parameter of <code>PopulateListView</code> is unused.
D7	<code>to_json(Action)</code>	Unused (the app never writes actions); harmless.
D8	<code>#include <optional>, <fstream>, <sstream></code>	No longer used since <code>findAction</code> replaced the <code>optional</code> -returning lookups.

5.3 Build and robustness notes

#	Item	Recommendation
R1	<code>#define UNICODE / _UNICODE</code> in the source <i>and</i> <code>/DUNICODE</code> on the command line	Causes warning C4005 (macro redefinition). Wrap in <code>#ifndef UNICODE</code> .
R2	<code>toupper</code> and <code>uintptr_t</code> rely on transitive includes	Add <code><cwctype></code> and <code><cstdint></code> .
R5	Stale comment in <code>OpenJsonFileForEditing</code> ("in case the exe lives under a directory with spaces")	The files are now under <code>%USERPROFILE%</code> ; update the wording.
R8	Window position and size are not remembered	Save them to a small settings file in the data folder (store DPI-independent values).
R9	Window title says "doubleclick a lineitem"	Consider "double-click a row or press Enter to launch".
R10	Initial window position	The window is scaled after <code>CW_USEDEFAULT</code> placement; on a small, high-DPI screen it may extend past the work area. Clamp to the monitor work area (<code>MonitorFromWindow / GetMonitorInfoW</code>).
R11	Manifest vs <code>.rc</code>	If the resource script already embeds a manifest, the pragma plus <code>/MANIFEST:EMBED</code> causes a duplicate-resource error; keep only one manifest source.

5.4 Security note

DDLaunch+ runs whatever `command` the JSON says, with the current user's rights. That is by design, and the files sit in the user's own profile, so the risk is the same as a user shortcut. Don't point DDLaunch+ at config files in shared or writable-by-others locations.

6. Version history

Version	Changes
2.1.0	Keyboard navigation (<code>IsDialogMessageW</code> , tab order, focus restore, Enter launches); Common Controls v6 manifest dependency; Per-Monitor V2 DPI awareness with live rescaling; buttons sized to their text; minimum window size (fixes R4); sharper icons
2.0.3	B3: clear messages for a missing type, an unknown type, a case-mismatched action name, an empty command, and failed launches (<code>ShellExecuteExW</code> + Win32 error text); single <code>findAction()</code> lookup
2.0.2	B4: generic RFC 3986 URL-scheme detection replaces the hard-coded list; fixes <code>file:///</code>
2.0.1	B1: reload only when a file's timestamp or size changes (no error-box loop, single load at startup); B2: sort order re-applied after reload (stable sort)
2.0.0	Initial documented release

7. Suggested next steps

1. Make action lookup case-insensitive (B5).
 2. Preserve the selection across reloads and sorts (B9).
 3. Keep the old device list when `connections.json` fails to load, and use `MessageBoxW` for JSON errors (B8, B10).
 4. Remove the dead code and unused includes in section 5.2.
 5. Remember window size and position, clamped to the monitor work area (R8, R10).
-

Documentation generated 09/25/2026 from `DDLlaunch.cpp` v2.1.0.